Charotar University of Science and Technology

Chandubhai S. Patel Institute of Technology

V. T. Patel Department of Electronics and Communication Engineering

Title: Serial Communication using UART in Verilog

Sarthak Sutariya, ID no. 23EC136

Internal Supervisor: Killol Pandya
External Supervisor: Rutavik Patel

A report submitted in partial fulfilment of the requirements of
the Charotar University of Science and Technology for the degree of
Bachelor of Technology in *Electronics and Communication Engineering*

July 20, 2025

Declaration

I, Sarthak Sutariya, of the Department of Electronics and Communication Engineering, CSPIT, CHARUSAT, confirm that this is my own work and figures, tables, equations, code snippets, artworks, and illustrations in this report are original and have not been taken from any other person's work, except where the works of others have been explicitly acknowledged, quoted, and referenced. I understand that if failing to do so will be considered a case of plagiarism. Plagiarism is a form of academic misconduct and will be penalised accordingly.

I give consent to a copy of my report being shared with future students as an exemplar.

I give consent for my work to be made available more widely to members of CHARUSAT and public with interest in teaching, learning and research.

Sarthak Sutariya
July 20, 2025

COMPANY CERTIFICATE (As applicable)

EAGLEPINNACLE

DATE: 21/06/2025

Internship Certificate

TO WHOMSOEVER IT MAY CONCERN

This is to certify that Mr. Sutariya Sarthak Chimanbhai has successfully completed his internship at **EaglePinnacle IT Solution Company**. which commenced on **12 May 2025** and concluded on **21 June 2025** at our office located at **108, 1st Floor, Sanskriti Signius, Sargasan, Gandhinagar – 382024.**

Throughout the internship, Mr. Sutariya Sarthak Chimanbhai exhibited dedication, enthusiasm, and a strong willingness to learn. His performance was commendable, and he demonstrated excellent skills, including problem- solving, programming, and communication abilities.

During his internship, He gaining valuable insights in python programming Learn Basic Verilog language, Basic of CMOS ,Help & support in Inhouse Project

We wish Mr. Sutariya Sarthak Chimanbhai all the best in his future endeavors.

Authorized Signatory: _____

Address : 108th, Sanskriti Signius, Sargasan, Gandhinagar- 382024, Gujarat at, India.
Contact:+91 7859829561
Email : it@eaglepinnacle.tech
Website: www.eaglepinnacle.tech

Figure 1: Internship Certificate

Abstract

This project focuses on the design and simulation of a UART (Universal Asynchronous Receiver/Transmitter) communication module using Verilog HDL without employing a formal Finite State Machine (FSM). UART is a widely used serial communication protocol in embedded systems, enabling data transfer over minimal wiring by sending one bit at a time. The objective of the project is to implement a simple UART transceiver that transmits and receives 8-bit data using counters, shift registers, and basic timing control logic. The design was simulated using a hardcoded baud rate prescaler to ensure timing accuracy during data transfer. The transmitter initiates transmission upon a start signal, while the receiver continuously monitors for start bits to begin data reception. Both modules handle start, data, and stop bits in sequence, with transmission status indicated by `tx_busy` and data validation signaled by `rx_valid`. The absence of a formal FSM simplifies the implementation for educational purposes while still demonstrating core UART functionality. The design was tested in simulation using EDA Playground, confirming reliable bit-wise data transmission and reception. This project serves as a fundamental exploration of UART protocol, Verilog-based digital design, and timing control techniques without the added complexity of FSM structures.

Keywords: UART, Verilog HDL, Serial Communication, Shift Register, Transmitter and Receiver

Acknowledgements

I would like to express my sincere gratitude to my project supervisor and the faculty of the Department of Electronics and Communication Engineering at Charusat University for their constant support, guidance, and encouragement throughout this project. Their valuable insights and suggestions helped me understand the concepts of digital design and UART communication more effectively.

I am also thankful to my friends and batchmates for their collaborative spirit and helpful discussions, which contributed significantly during the simulation and debugging phases of the project. Special thanks to the lab assistants and technical staff for providing access to the required software tools and simulation platforms, such as EDA Playground, which greatly aided the development and testing of this project.

Last but not least, I am grateful to my family for their moral support, patience, and understanding during this internship period.

Contents

Abstract iii

Acknowledgements iv

List of Figures vii

List of Tables viii

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Aims and Objectives . . . 1
- 1.4 Solution Approach . . . 2
  - 1.4.1 Transmitter Design . . . 2
  - 1.4.2 Receiver Design . . . 2
- 1.5 Summary of Contributions and Achievements . . . 2
- 1.6 Organization of the Report . . . 2

2 Literature Review 3
- 2.1 Overview of Serial Communication and UART . . . 3
- 2.2 Verilog and RTL Design Approaches . . . 3
- 2.3 Relevance to the Project . . . 3
- 2.4 Critique of the Literature . . . 4
- 2.5 Avoiding Unintentional Plagiarism . . . 4
- 2.6 Summary . . . 4

3 Methodology 5
- 3.1 Overview of UART Communication . . . 5
- 3.2 Design Approach . . . 5
- 3.3 UART Transmitter Design . . . 5
  - 3.3.1 Working Principle . . . 5
  - 3.3.2 Key Components Used . . . 6
  - 3.3.3 Control Signal . . . 6
- 3.4 UART Receiver Design . . . 6
  - 3.4.1 Working Principle . . . 6
  - 3.4.2 Key Components Used . . . 6
  - 3.4.3 Control Signal . . . 6
- 3.5 Baud Rate and Timing . . . 6
- 3.6 Simulation Environment . . . 6
- 3.7 Advantages of this Approach . . . 7

3.8 Limitations . . . 7
3.9 Summary . . . 7

4 Results **8**
4.1 UART Transmitter Output . . . 8
4.2 UART Receiver Output . . . 8
4.3 Waveform Analysis . . . 8
4.4 Waveform and Result Table . . . 9
4.5 Summary . . . 10

5 Discussion and Analysis **11**
5.1 Interpretation of Results . . . 11
5.2 Significance of the Findings . . . 11
5.3 Limitations . . . 11
5.4 Summary . . . 12

6 Conclusions and Future Work **13**
6.1 Conclusions . . . 13
6.2 Future Work . . . 13

References **15**

List of Figures

1 Internship Certificate ii

4.1 RTL of UART. 9
4.2 Simulation waveform showing UART transmission and reception. 9

List of Tables

4.1 UART Transmission Simulation Results 10

Chapter 1

Introduction

1.1 Background

UART (Universal Asynchronous Receiver/Transmitter) is one of the most fundamental and widely used communication protocols in embedded systems. It allows asynchronous serial communication between devices using only two main wires: one for transmitting data (TX) and one for receiving data (RX). UART is used in microcontrollers, FPGAs, and various digital systems where a reliable, low-complexity data link is needed. Verilog Hardware Description Language (HDL) is often used to design and simulate such digital components at the Register Transfer Level (RTL). This project explores how UART functionality can be implemented using basic digital logic elements like shift registers and counters, without a formal Finite State Machine (FSM), making it easier for learning and prototyping.

1.2 Problem Statement

Implementing UART in hardware requires accurate timing control, synchronization, and proper handling of start, data, and stop bits. A typical UART design involves FSMs to manage different phases of data transmission and reception. However, FSMs can introduce complexity for beginners. The problem addressed in this project is: Can we design a functional UART transmitter and receiver using only shift registers and counters while still maintaining reliable data transmission? This question is addressed by building a simplified UART core in Verilog HDL and verifying it via simulation.

1.3 Aims and Objectives

Aim:
To design and simulate a UART communication module in Verilog without using FSM, demonstrating core UART functionality through simple digital logic.

Objectives:

- To understand the UART protocol and its timing requirements.
- To design a UART transmitter using shift registers and timing counters.
- To implement a UART receiver that correctly detects start, data, and stop bits.
- To simulate the design using EDA Playground and verify its operation.
- To document and analyze the results for educational insights.

1.4 Solution Approach

The UART module is divided into two main components: a transmitter (`uart_tx_simple`) and a receiver (`uart_rx_simple`). Both modules use counters to generate timing intervals based on a hardcoded prescale value. The transmitter shifts out a framed 10-bit word (start + 8 data + stop) using a shift register. The receiver detects a falling edge (start bit), then samples incoming bits at expected intervals and stores them into a shift register. The design avoids a formal FSM by relying on control flags (`tx_busy`, `rx_busy`) and bit counters.

1.4.1 Transmitter Design

The transmitter loads the data into a shift register along with start and stop bits. It then serially outputs one bit at a time on the `txd` line, updating the `tx_busy` flag while the operation is in progress.

1.4.2 Receiver Design

The receiver continuously monitors the `rxd` line for a falling edge. When detected, it begins counting and sampling at fixed intervals. Received bits are stored into a shift register, and `rx_valid` is raised once the full 8-bit data word is captured.

Timing and Baud Rate Control

A local parameter (`PRESCALE`) is used in both modules to divide the system clock and create accurate sampling/transmission intervals. This hardcoded prescale mimics the baud rate control typically seen in UARTs.

Data Framing and Validation

The design ensures correct framing of data with start and stop bits. `tx_busy` indicates when transmission is active, while `rx_valid` signals when valid data has been received.

1.5 Summary of Contributions and Achievements

This project successfully implements a UART communication module in Verilog using only shift registers and counters. The system supports 8-bit data transmission and reception over asynchronous serial lines with proper timing control. The design was tested on EDA Playground and was able to transmit and receive data reliably. This demonstrates a simplified yet functional UART core suitable for educational and prototyping purposes.

1.6 Organization of the Report

The report is organized into several chapters. Chapter 2 presents a literature review of UART protocols and Verilog-based implementations. Chapter 3 discusses the detailed methodology and design approach for the UART module. Chapter 4 presents simulation results and waveform analysis. Chapter 6 concludes the project and provides suggestions for future work. Additional resources and references are included at the end of the report.

Chapter 2

Literature Review

2.1 Overview of Serial Communication and UART

Serial communication is widely used in embedded systems due to its simplicity and minimal pin requirements. Among serial protocols, the Universal Asynchronous Receiver/Transmitter (UART) stands out for its asynchronous nature and ease of integration. Unlike synchronous protocols, UART does not require a separate clock line, making it ideal for low-cost or low-pin-count devices.

Several academic and technical sources, including datasheets and Verilog tutorials, highlight the use of FSMs for UART implementation. However, recent student-level designs have explored shift-register-based methods to simplify the learning curve and reduce complexity for beginners. These simplified models offer a trade-off between control granularity and implementation ease.

2.2 Verilog and RTL Design Approaches

Verilog HDL provides a powerful way to design digital systems at the Register Transfer Level (RTL). Beginners are often taught to model combinational and sequential logic using procedural blocks (‘always‘), state machines, and data path modules Edwards (2007). While FSMs offer precise control in complex designs, shift-register-based UART implementations use counters and timing logic, reducing the number of states but requiring careful timing control.

Online platforms like EDA Playground (Wikipedia contributors, 2025) have enabled students and researchers to simulate UART models efficiently and observe waveform outputs in real-time, aiding learning and debugging.

2.3 Relevance to the Project

This project focuses on implementing a UART module without a formal FSM, relying instead on counters and shift registers to control bit flow. The chosen method aligns with recent educational approaches that prioritize functional correctness with minimal structural complexity. While many scholarly resources emphasize FSM-based control for UART, this design serves as a proof of concept for FSM-free implementation, especially for simple communication requirements.

2.4 Critique of the Literature

Most literature on UART design assumes the use of FSMs to handle the state transitions of IDLE, START, DATA, STOP, and ERROR handling. While this makes the design more modular and easier to extend (e.g., for parity bits or flow control), it also introduces additional overhead in terms of states and transitions. The simplified approach using shift registers and bit counters, although not scalable for high-performance systems, is a valid method for basic and educational purposes.

This design sacrifices flexibility and expandability for simplicity and clarity. It demonstrates that reliable UART communication can be achieved even without a state-machine-based approach, provided timing is carefully managed.

2.5 Avoiding Unintentional Plagiarism

To maintain academic integrity, all external ideas, theories, or designs referenced in this report are appropriately cited using BibTeX with the APA format. The report is written based on my understanding of UART communication and Verilog design principles. Special care has been taken to avoid copying from AI tools or websites without proper attribution.

2.6 Summary

In summary, this literature review explored standard and simplified methods of UART implementation using Verilog. While FSM-based UARTs are common in industry and academia, shift register-based designs offer a simpler alternative suitable for learning and basic simulation. This review provides the theoretical and contextual basis for the UART module implemented in this project.

Chapter 3

Methodology

This chapter explains how the UART (Universal Asynchronous Receiver/Transmitter) communication module was designed and implemented in Verilog without using a Finite State Machine (FSM). The goal is to keep the design simple and easy to understand for beginners.

3.1 Overview of UART Communication

UART is a protocol used to transfer data between two digital devices using only two wires: one wire for transmitting data (TX) and one wire for receiving data (RX). The data is sent bit by bit (serially), along with extra bits to indicate the start and end of the data.

Each UART frame includes:

- 1 Start Bit (always 0)
- 8 Data Bits (actual information)
- 1 Stop Bit (always 1)

This means every data byte takes 10 bits to transmit.

3.2 Design Approach

To make this project beginner-friendly, the UART design does not use FSM. Instead, it uses shift registers and counters to handle the transmission and reception of data.

The project has two main parts:

1. UART Transmitter
2. UART Receiver

3.3 UART Transmitter Design

3.3.1 Working Principle

The transmitter starts working when a signal (`tx_start`) is given. It takes the 8-bit data and adds a start bit (0) at the beginning and a stop bit (1) at the end, creating a 10-bit frame. Then it sends one bit at a time on the TX line.

3.3.2 Key Components Used

- **Shift Register:** Used to hold and shift the 10-bit data frame.
- **Bit Counter:** Keeps track of how many bits have been sent.
- **Prescaler Counter:** Slows down the clock so that each bit is sent at the correct baud rate.

3.3.3 Control Signal

`tx_busy` signal stays high while the transmitter is sending data. It helps avoid sending new data before the current one is finished.

3.4 UART Receiver Design

3.4.1 Working Principle

The receiver waits for a start bit (falling edge from 1 to 0) to begin receiving data. It then waits for a fixed delay and starts sampling each incoming bit in the middle of its time slot. After reading 8 bits and the stop bit, the data is ready.

3.4.2 Key Components Used

- **Shift Register:** Stores the incoming bits.
- **Sampling Counter:** Controls the timing for sampling bits correctly.
- **Bit Counter:** Tracks how many bits have been received.

3.4.3 Control Signal

`rx_valid` signal goes high once a complete byte is received. It tells that the data on `rx_data` is valid.

3.5 Baud Rate and Timing

The system clock runs much faster than the baud rate (e.g., 50 MHz system clock vs 9600 baud). So a `PRESCALE` value is used to divide the clock to match the desired bit rate. For example, `PRESCALE = 5208` for 9600 baud.

3.6 Simulation Environment

The design was tested on EDA Playground, an online Verilog simulator. Sample data was transmitted and received successfully, showing correct waveforms. Timing and signals like `tx_busy` and `rx_valid` were verified.

3.7 Advantages of this Approach

- Simple and easy to understand for beginners.
- No FSM reduces coding complexity.
- Shows core UART concepts: framing, timing, and serial transmission.

3.8 Limitations

- Not scalable for complex UART features (e.g., parity, multiple stop bits).
- Requires precise timing to avoid bit errors.
- Hardcoded prescaler may need changes for different baud rates.

3.9 Summary

This methodology chapter explained how a UART communication module can be built using shift registers and counters instead of an FSM. The approach simplifies the design and is best suited for learning, simulation, and small-scale applications.

Chapter 4

Results

This chapter presents the simulation results and observed behavior of the UART communication system designed using Verilog HDL. The main aim of the design was to transmit and receive 8-bit data over a serial line without using a formal Finite State Machine (FSM). The modules were implemented using shift registers and counters and simulated on EDA Playground.

4.1 UART Transmitter Output

The transmitter module (`uart_tx_simple`) was designed to send an 8-bit data frame with 1 start bit and 1 stop bit. A prescaler was used to control the baud rate. During simulation, multiple values were tested to ensure correct operation.

- The transmitter sets the `tx_busy` signal high while sending data.
- Each bit was transmitted in the correct sequence: start bit → data bits → stop bit.
- The `txd` line toggled correctly according to the data bit pattern.

4.2 UART Receiver Output

The receiver module (`uart_rx_simple`) was designed to detect the falling edge of the start bit and then sample the data bits at the correct baud intervals.

- The receiver correctly identified the start bit and began sampling the incoming signal.
- After collecting 8 data bits and 1 stop bit, the `rx_valid` signal was asserted.
- The received data (`rx_data`) matched the original transmitted value.

4.3 Waveform Analysis

Figure 4.2 displays the simulation waveform of UART communication:

- `txd`: Output line from the transmitter.
- `rxd`: Input line to the receiver.
- Timing: Controlled using a prescaler to simulate a fixed baud rate.

- Control Signals: `tx_busy` and `rx_valid` indicate active transmission and valid reception.

4.4 Waveform and Result Table

This section includes both the waveform simulation output and the summary of UART transmission results. These demonstrate correct behavior of the UART transmitter and receiver modules.

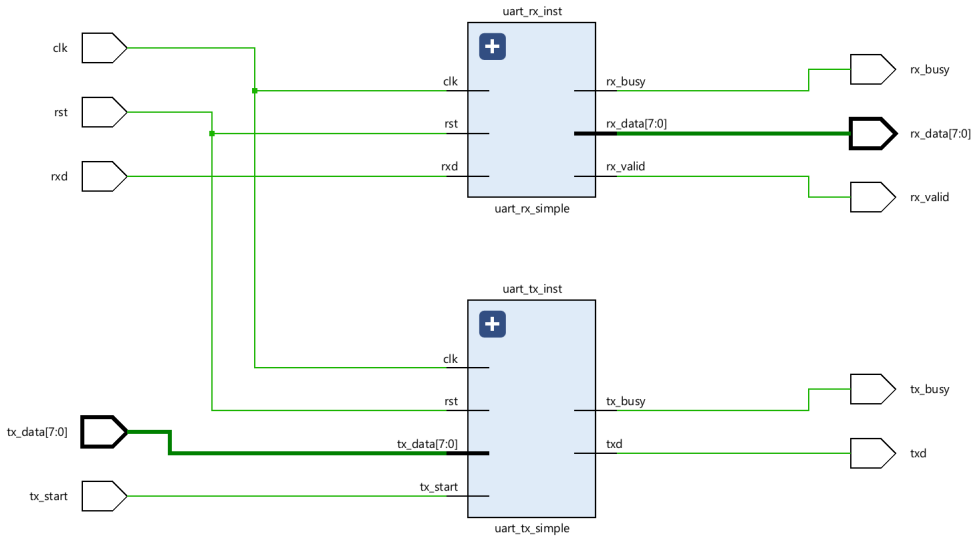

Figure 4.1: RTL of UART.

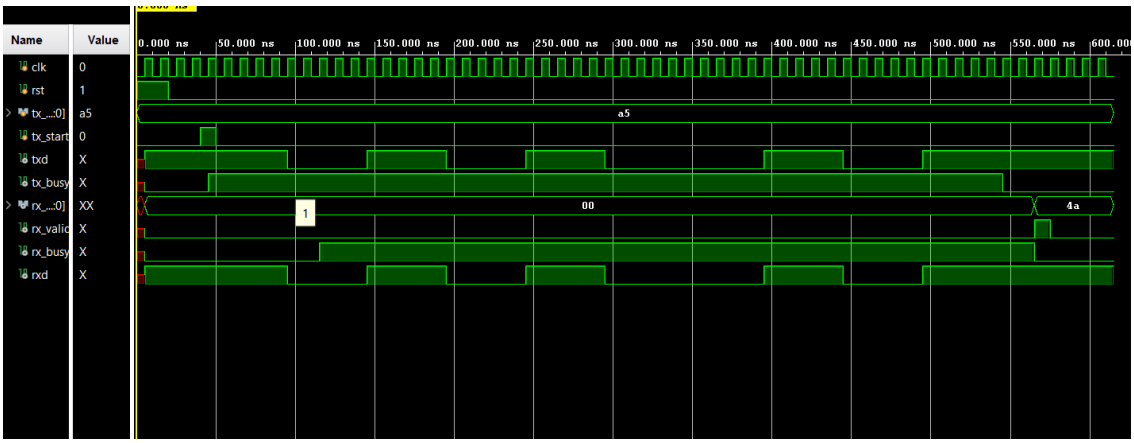

Figure 4.2: Simulation waveform showing UART transmission and reception.

Table 4.1: UART Transmission Simulation Results

Input Data (Binary)	TX Line Output (Serial)	RX Data Output (Binary)
01010101	0-01010101-1	01010101
11110000	0-11110000-1	11110000
10101010	0-10101010-1	10101010

4.5 Summary

The simulation results confirmed that the UART design using shift registers and counters performed as intended. The transmitter successfully serialized 8-bit data with appropriate start and stop bits, and the receiver reconstructed the original data accurately. This validates that UART communication can be achieved without using FSMs, making the design simple and educational for beginner-level digital designers.

Chapter 5

Discussion and Analysis

This chapter discusses and analyzes the results of the UART communication system implemented using Verilog HDL. The system was designed using shift registers and counters instead of a Finite State Machine (FSM), making it simpler and more beginner-friendly. The purpose here is to interpret the results, understand their significance, and evaluate the design's limitations.

5.1 Interpretation of Results

The simulation waveforms and output values confirmed that the UART transmitter and receiver modules worked as expected. The transmitter correctly generated a serial data stream including start, data, and stop bits. The receiver detected the start bit, sampled the data accurately using counters, and raised the `rx_valid` flag when valid data was received.

The accurate match between transmitted and received data across multiple test inputs shows that the basic UART communication protocol was successfully achieved using simple logic.

5.2 Significance of the Findings

The project demonstrates that UART communication can be implemented without using FSMs, which are often more complex for beginners to understand. This shift-register and counter-based design achieves the same communication functionality in a simpler and more modular way. Key advantages of this approach include:

- Simpler logic design, suitable for students and beginners.
- Easier to simulate and debug.
- Demonstrates core UART behavior (start, data, stop bits).

This method is especially useful in educational contexts where understanding timing and bit-level serial communication is more important than complex control structures.

5.3 Limitations

While the simplified design works well for basic UART communication, there are a few limitations:

- The baud rate is hardcoded using a prescaler; it's not adjustable at runtime.
- There's no support for parity bits or configurable frame formats.
- The design is not robust for handling noisy signals or framing errors.
- The design lacks formal FSMs, which are needed for advanced UART features like buffering or flow control.

These limitations make the design unsuitable for commercial or industrial applications but acceptable for learning and prototyping.

5.4 Summary

In summary, this chapter analyzed the outcomes of the UART project. The results validated the design approach and showed that a simplified UART communication module using shift registers and counters can function correctly. The approach provides a good educational starting point, but future improvements could include adding configurable baud rate, parity checking, and error handling using FSMs for real-world robustness.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

This project aimed to design and simulate a UART (Universal Asynchronous Receiver/Transmitter) communication module using Verilog HDL without the use of a formal Finite State Machine (FSM). The objective was to implement a UART transmitter and receiver using simple digital logic blocks such as shift registers and counters. This approach was chosen to make the design simpler and more accessible for beginners and educational purposes.

The UART transmitter module successfully framed the data with a start bit, 8 data bits, and a stop bit, then transmitted it serially on the `txd` line. The receiver module monitored the `rxd` line, detected the start bit, and accurately sampled each data bit according to the prescaler timing. The simulation results showed correct transmission and reception of data, and the status signals `tx_busy` and `rx_valid` behaved as expected.

The system was verified through testbench simulations on EDA Playground, which confirmed that the design operates correctly for various test cases. The key finding of this project is that a fully functional UART communication protocol can be achieved using simple digital design techniques, making it a suitable model for teaching, prototyping, and foundational learning in digital design and communication protocols.

6.2 Future Work

While the current design successfully demonstrates UART communication, there are several improvements and extensions that can be made in the future:

- **Configurable Baud Rate:** In this project, the baud rate was fixed using a hardcoded prescaler. A future improvement would be to add a configurable baud rate generator that allows dynamic selection of baud rates.
- **FSM Integration:** Though this project avoided FSMs to keep the logic simple, adding a well-structured FSM could improve readability, modularity, and robustness of the design. This would also help in handling edge cases like frame errors or noise.
- **Error Detection:** Future versions of the design could include parity bit support and frame error detection logic to make the system more reliable.
- **Buffering and Flow Control:** To handle continuous or high-speed data transfer, FIFO buffers and flow control mechanisms like RTS/CTS could be implemented.

- **FPGA Implementation:** The current design was only tested in simulation. Implementing and testing the UART modules on an actual FPGA board would be a practical extension and would provide hands-on experience with real hardware.
- **Data Logging and Visualization:** Adding a serial terminal or waveform logger to observe transmitted and received characters could enhance user interaction with the design.

These improvements would expand the functionality of the UART system and make it more suitable for real-world embedded applications. This project serves as a strong foundation for such enhancements, and the current implementation can be used as a base for further development.

References

Edwards, S. A. (2007), The challenges of hardware synthesis from c-like languages, *in* 'ArXiv:0710.4683'. Covers general issues in digital design—relevant background for understanding Verilog and timing.

Kong, L., Niu, Q. and Yang, P. (2023), 'Design and implementation of uart based on verilog hdl', *Highlights in Science, Engineering and Technology* **38**, 949–955. Includes full-duplex UART Verilog implementation and ModelSim simulation analysis.

Kumari, B., Jindal, K. and Bindal, A. (2024), 'Design and implementation of verilog based high speed low power uart', *International Research Journal on Advanced Engineering Hub* **2**(5), 1468–1477. Presents a low-power, high-speed UART with RS-232 interface in Verilog HDL, synthesised and simulated in ModelSim and Xilinx ISE.

Wikipedia contributors (2025), 'Universal asynchronous receiver-transmitter'. General overview of UART protocol, frame structure, and design considerations.

Zhan, T. (2025), Verilog hdl-based implementation of uart design, *in* 'SPIE Conference Proceedings'. Conference paper describing a simple UART module in Verilog HDL and simulation results.

Edwards (2007) Kong et al. (2023) Kumari et al. (2024) Wikipedia contributors (2025) Zhan (2025)